Technical Approach Report: AI Ticket Triage and Runbook Copilot for Cybit

Companion to the AI Opportunity Report and the proof-of-concept code (`cybit_triage_poc.zip`)

Important framing. The brief names "Cynbit"; I assumed Cybit (cybit.com). The prototype uses **only synthetic data** that I generated, so every metric below shows that the pipeline works and where it is weak. None of them predicts production accuracy at Cybit.

1. What the system does

Given a ticket or alert text, the system returns:

1. a **category** (10 classes, such as account access, phishing, endpoint alert, network, cloud, backup, Microsoft 365, hardware, data and BI, access change);
2. a **priority** (P1-P4) with a confidence score;
3. the **top three runbooks**, retrieved from a knowledge base;
4. a **summary, numbered next steps and a draft customer reply**, grounded in those runbooks;
5. a **human-review flag** with the reasons it fired.

The output is a suggestion for an engineer. Nothing is sent to a customer automatically.

2. Architecture

Stage	Component	Why
1. Ingest	Ticket text (title plus description)	Single text field keeps the prototype simple
2. Classify category	TF-IDF (word 1-2 grams plus character 3-5 grams) into logistic regression, **fused with** similarity to a domain lexicon and runbook text	Fast, cheap, auditable; character n-grams tolerate typos; the lexicon gives a cold-start prior
3. Classify priority	Same text features plus 9 hand-built **severity flags** (active attack, wide scope, production impact, urgency wording, request wording and so on) into logistic regression	Priority depends on *scope and impact*, not topic, so explicit signals help with little data

Stage	Component	Why
4. Retrieve	TF-IDF cosine similarity over runbooks, plus a boost for the predicted category's runbook	Transparent and testable; easy to swap for embeddings
5. Draft	LLM (Gemini API via the google-genai SDK, model set by `TRIAGE_MODEL`, default `gemini-3.8-flash`) writes JSON from the retrieved runbooks only. Falls back to a deterministic template if no key or on any error	Grounded generation; the demo always runs offline
6. Gate	Human review if confidence is below a threshold, if priority is P1, or if alarming keywords appear on a P3/P4 ticket	Protects against the costly failure: a real incident treated as routine
7. UI	Streamlit app showing category, priority, confidence, sources and review reasons	Lets a reviewer see *why*

Why a classical model plus an LLM, not an LLM for everything?

- **Cost and latency:** classification runs in milliseconds with no per-ticket API cost, which matters at ticket volume.
- **Auditability:** logistic regression weights and keyword flags can be inspected and explained to a customer or auditor.
- **Data control:** classification can run entirely inside Cybit's environment. Only the drafting step needs an external model, and it can be switched off.
- **Honest limitation:** a classical model needs varied training data. With real ticket history this is easy; with 1,500 template-generated tickets it is not (see section 5). In production I would benchmark it against embedding-based and LLM few-shot classifiers.

3. Data pipeline

Prototype (what I built)

- `knowledge.py`: 10 synthetic runbooks (5 steps each), category list, priority definitions and a domain lexicon.

- `data_gen.py` : generates 1,500 tickets from templates with prefixes, suffixes, typos, 7% priority label noise and 4% category label noise, so the task is not trivially clean.
- `stress_set.py` : 30 hand-written tickets with wording not in the templates. I used this as a **development** set.
- `holdout_set.py` : 24 further hand-written tickets, written after the first evaluation round and **never used for tuning**.

Production pipeline (what I would build)

1. **Export** historical tickets from the PSA/ITSM tool, with engineer-confirmed category, priority, resolution and time-to-resolve.
2. **Redact** personal data and secrets (names, emails, IPs, credentials) before storage or any model call.
3. **Clean and deduplicate**; remove auto-generated boilerplate; keep the original language.
4. **Label quality check:** audit a sample for inconsistent engineer labels, because priority labels are usually noisy.
5. **Split by time and by customer**, not randomly, to avoid leakage between near-duplicate tickets.
6. **Index runbooks** by chunk (per step or per section) with metadata for tool and customer.
7. **Feedback loop:** log every suggestion and every engineer correction as new training data.

4. Prompt design (drafting step)

- **System instruction:** use only the supplied runbook excerpts; never invent commands, tools or policies; if the runbooks do not cover the issue, say so and return no steps.
- **Structured output:** JSON with `summary` (at most two sentences), `next_steps` (short strings citing runbook IDs) and `customer_reply` (polite, no technical promises, no estimated time).
- **Context:** ticket text, the predicted category and priority, and the top three runbooks.
- **Validation:** parse the JSON; reject and fall back to the template if parsing fails; also reject the draft if it cites a runbook ID that does not exist (implemented; tested with a mocked client).
- **Injection defence:** ticket text is passed as data, the model has no tools, and its output goes to a human first.

LLM step status: The LLM drafting path was run end to end with the Gemini API in the Streamlit demo and returned grounded drafts (labelled "Drafted by: llm"); during testing the API also returned temporary 503 overload errors, which the app now retries automatically. This was a manual check on a few tickets, and I did not **measure draft quality or groundedness systematically**. All metrics in this report come from the classification, retrieval and gating logic, which is what runs when the template drafter is used.

5. Results

All numbers come from `evaluate.py` (fixed random seeds). The majority-class baseline scores 11.2% on category and 34.9% on priority.

Test	n	Category acc.	Priority exact	Priority within 1 level	P1 recall
Random 25% split of synthetic data	375	97.9%	92.8%	99.7%	95.6%
Dev stress set, first model (TF-IDF plus logistic regression only)	30	46.7%	40.0%	86.7%	20.0%
Dev stress set, final model	30	86.7%	56.7%	96.7%	80.0%
Clean hold-out, final model	24	**100%**	**75.0%**	**87.5%**	**100%**

Retrieval. On the clean hold-out the correct runbook was ranked first for 24 of 24 tickets. On the dev stress set, retrieval alone found it first for 60% and in the top three for 77%; with the classifier boost these rose to 87% and 93%.

Gating on the hold-out. All P1 tickets were flagged for human review. About 62.5% of tickets (15 of 24) passed without a review flag, and none of those had a wrong category. 37.5% were routed to a person.

What the results actually say

1. **The first model failed where it mattered.** It looked excellent on a random split (97.9%) but fell to 46.7% on tickets worded differently, and caught only 1 in 5 true P1s. A random split of template-generated data is a misleading test, because the same templates appear in training.
2. **Fixes that helped:** a domain-knowledge prior (lexicon and runbook text used as extra training signal and as a fusion component) and explicit severity features for priority.
3. **Remaining weakness: priority.** Exact priority accuracy on the hold-out is 75%. Of the six priority misses, five were over-escalations (for example a docking-station fault rated P1) and one was an under-escalation (a failed backup rated P4 instead of P2).

Over-escalation is annoying; under-escalation is the dangerous direction, and the review rules are built to catch it.

4. **Caveats on the hold-out.** It has only 24 tickets, so a perfect category score still leaves a wide uncertainty range (a 95% lower bound of roughly 86%). I also wrote the lexicon and the hold-out myself, so they share my vocabulary. Treat the hold-out as optimistic.

6. Production evaluation plan

- **Offline:** replay 6-12 months of historical tickets with a time-based split; report per-category precision and recall, priority confusion matrix and **P1 recall** (target near 100% with review).
- **Shadow mode:** run live without showing suggestions; compare with engineers' decisions for two weeks.
- **Assist mode:** measure acceptance and edit rate of drafts, time to first response, and SLA breaches against a control group of engineers or customers.
- **Monitoring:** weekly drift report (class mix, confidence distribution, override rate); alert if override rate rises.

7. Execution plan

Phase	Duration	Output
0. Discovery	1 week	Ticket export, data access and privacy review, runbook inventory
1. Offline baseline	2 weeks	Classifier and retrieval benchmarks on real history; decide classical, embedding or LLM classification
2. Shadow pilot	2 weeks	Live comparison, no customer impact
3. Assist pilot	4 weeks	Integration with the ticketing tool, feedback capture, drafted replies reviewed by engineers
4. Extend	ongoing	NOC and SOC alert sources, security validation reports, customer-facing assistant

Likely stack in production: Python service (FastAPI) behind the ticketing webhook, a vector store for runbooks, a private or contractually protected model endpoint, and a dashboard for metrics and review queues.

8. Limitations and next steps

- Synthetic data only; real tickets are longer, noisier and often multilingual.
- Small hand-written test sets; no confidence intervals beyond the rough bound above.
- LLM drafting untested here; needs a groundedness check (are all steps traceable to a runbook?).
- Retrieval is keyword-based TF-IDF; replacing it with hybrid keyword plus embedding search and a reranker is the first upgrade.
- Priority should eventually use structured context (customer SLA tier, asset criticality, number of affected users), not just text.

9. How to run the prototype

```
pip install -r requirements.txt
python evaluate.py
streamlit run app.py
# optional: export GEMINI_API_KEY=... for LLM-drafted replies
```